hello_

61.01

COLECCIÓN BACKEND

Python fácil desde cero (para humanos que aún no hablan con serpientes)



Aprende sintaxis limpia y lógica clara

Domina funciones, estructuras y módulos

Crea tus primeros programas reales paso a paso

Todo empezó con un print("Hola mundo") y hasta hoy ...

@mihifidem
Formador, desarrollador y
fiel creyente en el poder de a,b=b,a

Python

Introducción a la programación

Introducción

Este capítulo es una introducción a la programación para conocer, desde un enfoque sencillopero aclaratorio, los mecanismos que hay detrás de ello.

Indice

1.1 Hablando con la máquina
1.1.1 Código máquina
1.1.2 Ensamblador
1.1.3 C
1.1.4 Python
1.1.5 Compiladores

1.2 Algo de historia
1.2.1 Hitos de la computación
1.2.2 De los computadores a la programación
1.2.3 Cronología de lenguajes de programación
1.2.4 Creadores de lenguajes de programación

1.3 Python
1.3.1 Características del lenguaje
1.3.2 Uso de Python
1.3.3 Python2 vs Python3
1.3.4 Zen de Python



Consejo: «Programming is not about typing, it's about thinking.» – Rich Hickey

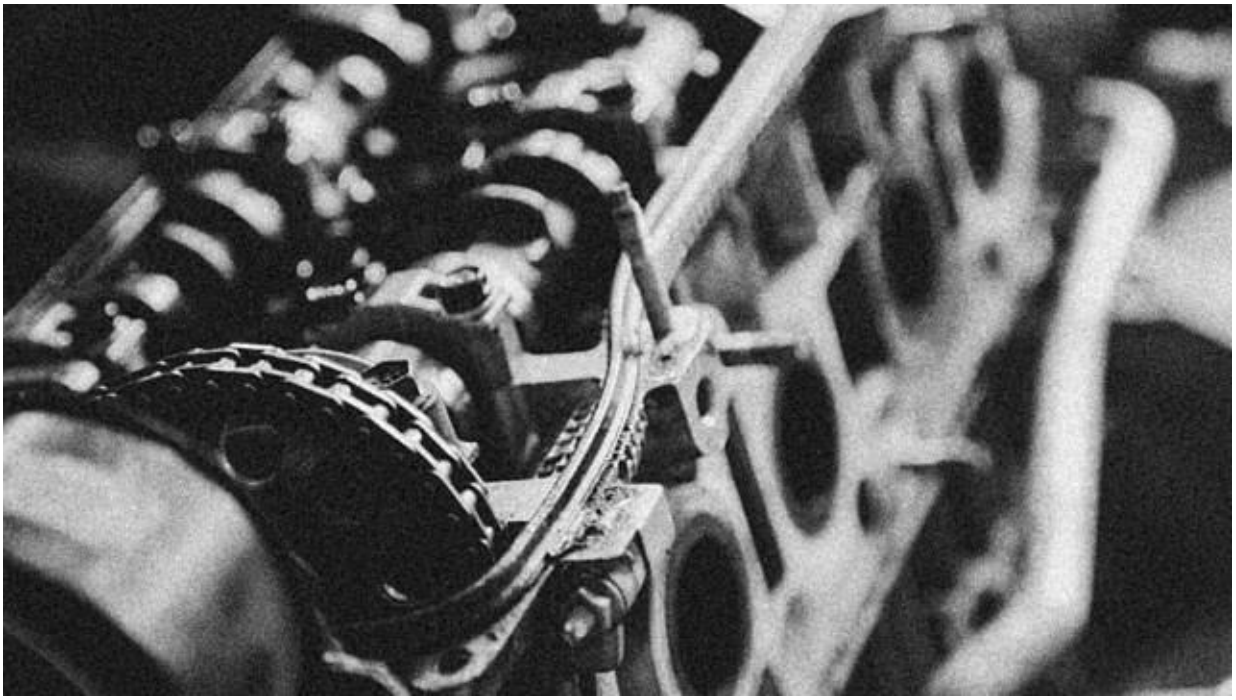
[1] En la foto de portada aparecen los Monty Python. Fuente: noticiascyl

CAPÍTULO 1

Introducción

Este capítulo es una introducción a la programación para conocer, desde un enfoque sencillopero aclaratorio, los mecanismos que hay detrás de ello.

1.1 Hablando con la máquina



Los ordenadores son dispositivos complejos pero están diseñados para hacer una cosa bien: **ejecutar aquello que se les indica**. La cuestión es cómo indicar a un ordenador lo que queremos que ejecute. Esas indicaciones se llaman técnicamente **instrucciones** y se expresan en un lenguaje. Podríamos decir que **programar** consiste en escribir instrucciones para que sean ejecutadas por un ordenador. El lenguaje que utilizamos para ello se denomina **lenguaje de programación**.[1]

1.1.1 Código máquina

Pero aún seguimos con el problema de cómo hacer que un ordenador (o máquina) entienda el lenguaje de programación. A priori podríamos decir que un ordenador sólo entiende un lenguaje muy «simple» denominado código máquina. En este lenguaje se utilizan únicamente los símbolos **0** y **1** en representación de los *niveles de tensión* alto y bajo, que al fin y al cabo, son los estados que puede manejar un circuito digital. Hablamos de sistema binario. Si tuviéramos que escribir programas de ordenador en este formato sería una tarea ardua, pero afortunadamente se han ido creando con el tiempo lenguajes de programación intermedios que, posteriormente, son convertidos a código máquina.

Si intentamos visualizar un programa en código máquina, únicamente obtendríamos una secuencia de ceros y unos:

```
00001000 00000010 01111011 10101100 10010111 11011001 01000000 01100010
00110100 00010111 01101111 10111001 01010110 00110001 00101010 00011111
10000011 11001101 11110101 01001110 01010010 10100001 01101010 00001111
11101010 00100111 11000100 01110101 11011011 00010110 10011111 01010110
```

1.1.2 Ensamblador

El primer lenguaje de programación que encontramos en esta «escalada» es **ensamblador**. Veamos un ejemplo de código en ensamblador del típico programa que se escribe por primeravez, el *«Hello, World»*:

```
SYS_SALIDA equ 1

section .data
    msg db "Hello, World",0x0a
    len equ $ - msg ;longitud de msg

section .text
global _start ;para el linker
_start: ;marca la entrada
    mov eax, 4 ;llamada al sistema (sys_write)
```

(Continue en la próxima página)

[1] Foto original por Garett Mizunaka en Unsplash.

(proviene de la página anterior)

```
    mov ebx, 1 ;descripción de archivo (stdout)mov ecx,
    msg ;msg a escribir
    mov edx, len ;longitud del mensaje
    int 0x80 ;llama al sistema de interrupciones

fin: mov eax, SYS_SALIDA ;llamada al sistema (sys_exit)int 0x80
```

Aunque resulte difícil de creer, lo «único» que hace este programa es mostrar en la pantalla de nuestro ordenador la frase «Hello, World», pero además teniendo en cuenta que sólo funcionará para una arquitectura x86.

1.1.3 C

Aunque el lenguaje ensamblador nos facilita un poco la tarea de desarrollar programas, sigue siendo bastante complicado ya que las instrucciones son muy específicas y no proporcionan una semántica entendible. Uno de los lenguajes que vino a suplir – en parte – estos obstáculosfue C. Considerado para muchas personas como un referente en cuanto a los lenguajes de programación, permite hacer uso de instrucciones más claras y potentes. El mismo ejemplo anterior del programa *«Hello, World»* se escribiría así en lenguaje *C*:

```
#include <stdio.h>

int  main() {
    printf("Hello, World");
    return 0;
}
```

1.1.4 Python

Si seguimos «subiendo» en esta lista de lenguajes de programación, podemos llegar hasta Python. Se dice que es un lenguaje de *más alto nivel* en el sentido de que sus instrucciones son más entendibles por un humano. Veamos cómo se escribiría el programa *«Hello, World»*en el lenguaje de programación Python:

```
print('Hello, World')
```

¡Pues así de fácil! Hemos pasado de *código máquina* (ceros y unos) a *código Python* en el que se puede entender perfectamente lo que estamos indicando al ordenador. La pregunta que surge es: ¿cómo entiende una máquina lo que tiene que hacer si le pasamos un programa hecho en Python (o cualquier otro lenguaje de alto nivel)? La respuesta es un **compilador**.

1.1.5 Compiladores

Los compiladores son programas que convierten un lenguaje «cualquiera» en *código máquina*.Se pueden ver como traductores, permitiendo a la máquina interpretar lo que queremos hacer.

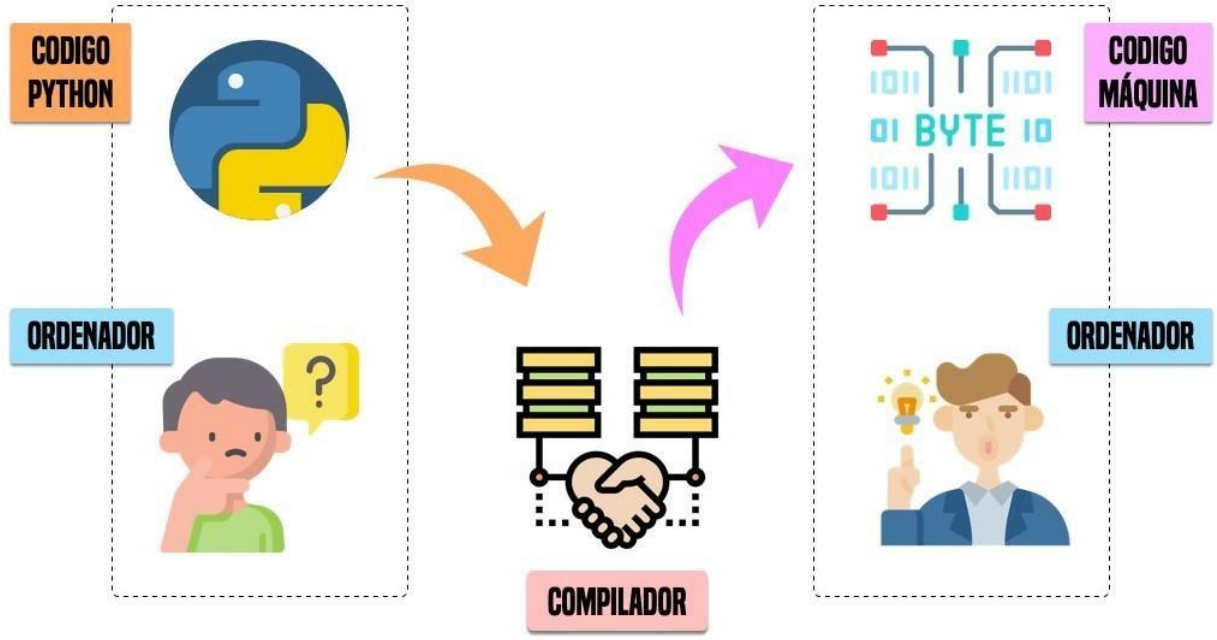


Figura 1: Esquema de funcionamiento de un compilador[2]

Nota: Para ser más exactos, en Python hablamos de un **intérprete** en vez de un compilador, pero a los efectos es prácticamente lo mismo. La diferencia está en que el intérprete realiza la «compilación» (*interpretación*) y la «ejecución» de una vez, mientras que el compilador genera un formato «ejecutable» (*código objeto*) que se ejecuta en otra fase posterior.

[2] Iconos originales por Flaticon.

1.2 Algo de historia



La historia de la programación está relacionada directamente con la aparición de los computadores, que ya desde el siglo XV tuvo sus inicios con la construcción de una máquinaque realizaba operaciones básicas y raíces cuadradas (Gottfried Wilheml von Leibniz); aunque en realidad la primera gran influencia hacia la creación de los computadores fue la máquina diferencial para el cálculo de polinomios, proyecto no concluido de Charles Babbage (1793-1871) con el apoyo de Lady Ada Countess of Lovelace (1815-1852), primera persona que incursionó en la programación y de quien proviene el nombre del lenguaje de programación ADA creado por el DoD (Departamento de defensa de Estados Unidos) en la década de 1970.[1]

1.2.1 Hitos de la computación

La siguiente tabla es un resumen de los principales hitos en la historia de la computación:

Tabla 1: Hitos en la computación

Personaje	Aporte	Año
Gottfried Leibniz	Máquinas de **operaciones básicas**	XV
Charles Babbage	**Máquina diferencial** para el cálculo de polinomios	XVII

Continue en la próxima página

[1] Foto original por Dario Veronesi en Unsplash.

Tabla 1 – proviene de la página anterior

Personaje	Aporte	Año
Ada Lovelace	Matemática, informática y escritora británica. **Primera programadora** de la historia por el desarrollo de algoritmos para la máquina analíticade Babbage	XVII
George Boole	Contribuyó al **algebra binaria** y a los sistemas de circuitos de computadora (álgebra booleana)	1854
Herman Hollerit	Creador de un **sistema para automatizar** la pesada tarea del censo	1890
Alan Turing	**Máquina de Turing** - una máquina capaz de resolver problemas - Aportes de Lógica Matemática - Computadora con tubos de vacío	1936
John Atanasoff	Primera **computadora digital** electrónica patentada: Atanasoff Berry Computer (ABC)	1942
Howard Aiken	En colaboración con IBM desarrolló el **Mark I**, una computadora electromecánica de 16 metros de largo y más de dos de alto que podía realizar las cuatro operaciones básicas y trabajar con informaciónalmacenada en forma de tablas	1944
Grace Hopper	Primera **programadora** que utilizó el Mark I	1945
John W. Mauchly	Junto a John Presper Eckert desarrolló una **computadora electrónica** completamente operacional a gran escala llamada Electronic Numerical Integrator And Computer (ENIAC)	1946
John Von Neumann	Propuso guardar en **memoria** no solo la **información**, sino también los **programas**, acelerando los procesos	1946

Luego los avances en las ciencias informáticas han sido muy acelerados, se reemplazaron lostubos de vacío por transistores en 1958 y en el mismo año, se sustituyeron por circuitos integrados, y en 1961 se miniaturizaron en **chips de silicio**. En 1971 apareció el primer microprocesador de Intel; y en 1973 el primer sistema operativo CP/M. El primer computadorpersonal es comercializado por IBM en el año 1980.

[2] Fuente: Meatze.



Figura 2: Ada Lovelace: primera programadora de la historia[2]

1.2.2 De los computadores a la programación

De acuerdo a este breve viaje por la historia, la programación está vinculada a la aparicióndel los computadores, y los lenguajes tuvieron también su evolución. Inicialmente, como ya hemos visto, se programaba en **código binario**, es decir en cadenas de 0s y 1s, que es el lenguaje que entiende directamente el computador, tarea extremadamente difícil; luego se creó el **lenguaje ensamblador**, que aunque era lo mismo que programar en binario, al estar en letras era más fácil de recordar. Posteriormente aparecieron **lenguajes de alto nivel**, que en general, utilizan palabras en inglés, para dar las órdenes a seguir, para lo cual utilizanun proceso intermedio entre el lenguaje máquina y el nuevo código llamado código fuente, este proceso puede ser un compilador o un intérprete.

Un **compilador** lee todas las instrucciones y genera un resultado; un **intérprete** ejecuta y genera resultados línea a línea. En cualquier caso han aparecido nuevos lenguajes de programación, unos denominados estructurados y en la actualidad en cambio los lenguajes orientados a objetos y los lenguajes orientados a eventos.[3]

1.2.3 Cronología de lenguajes de programación

Desde la década de 1950 se han sucedido multitud de lenguajes de programación que cada vez incorporan más funcionalidades destinadas a cubrir las necesidades del desarrollo de aplicaciones. A continuación se muestra una tabla con la historia de los lenguajes de programación más destacados:

El **número** actual de lenguajes de programación depende de lo que se considere un *lenguaje de programación* y a *quién se pregunte*. Según TIOBE más de 250; según Wikipedia más de 700, según Language List más de 2500; y para una cifra muy alta podemos considerar a Online Historical Encyclopaedia of Programming Languages que se acerca a los **9000**.

1.2.4 Creadores de lenguajes de programación

El avance de la computación está íntimamente relacionado con el desarrollo de los lenguajes de programación. Sus creadores y creadoras juegan un *rol fundamental* en la historia tecnológica. Veamos algunas de estas personas:[4]

Tabla 2: Creadores de lenguajes de programación

Personaje	Aporte
Alan Cooper	Desarrollador de **Visual Basic**
Alan Kay	Pionero en programación orientada a objetos. Creador de **Smalltalk**

Continue en la próxima página

[3] Fuente: Universidad Técnica del Norte.
[4] Fuente: Wikipedia.

Tabla 2 – proviene de la página anterior

Personaje	Aporte
Anders Hejlsberg	Desarrollador de **Turbo Pascal**, **Delphi** y **C#**
Bertrand Meyer	Inventor de **Eiffel**
Bill Joy	Inventor de **vi**. Autor de **BSD Unix**. Creador de **SunOS**, el cual se convirtió en **Solaris**
Bjarne Stroustrup	Desarrollador de **C++**
Brian Kernighan	Coautor del primer libro de programación en lenguaje **C** con Dennis Ritchie y coautor de los lenguajes de programación **AWK** y **AMPL**
Dennis Ritchie	Inventor de C. Sistema Operativo **Unix**
Edsger W. Dijkstra	Desarrolló las bases para la programación estructurada
Grace Hopper	Desarrolladora de **Flow-Matic**, influenciando el lenguaje **COBOL**
Guido van Rossum	Creador de **Python**
James Gosling	Desarrollador de **Oak**. Precursor de **Java**
Joe Armstrong	Creador de **Erlang**
John Backus	Inventor de **Fortran**
John McCarthy	Inventor de **LISP**
John von Neumann	Creador del concepto de sistema operativo
Ken Thompson	Inventor de B. Desarrollador de **Go**. Coautor del sistema operativo Unix
Kenneth E. Iverson	Desarrollador de **APL**. Co-desarrollador de **J** junto a Roger Hui
Larry Wall	Creador de **Perl** y Perl 6
Martin Odersky	Creador de **Scala**. Previamente contribuyó en el diseño de Java
Mitchel Resnick	Creador del lenguaje visual **Scratch**
Nathaniel Rochester	Inventor del primer lenguaje en **ensamblador** simbólico (IBM 701)
Niklaus Wirth	Inventor de **Pascal**, **Modula** y **Oberon**
Robin Milner	Inventor de **ML**. Compartió crédito en el método Hindley–Milner de inferencia de tipo polimórfica
Seymour Papert	Pionero de la inteligencia artificial. Inventor del lenguaje de programación **Logo** en 1968
Stephen Wolfram	Creador de **Mathematica**
Yukihiro Matsumoto	Creador de **Ruby**

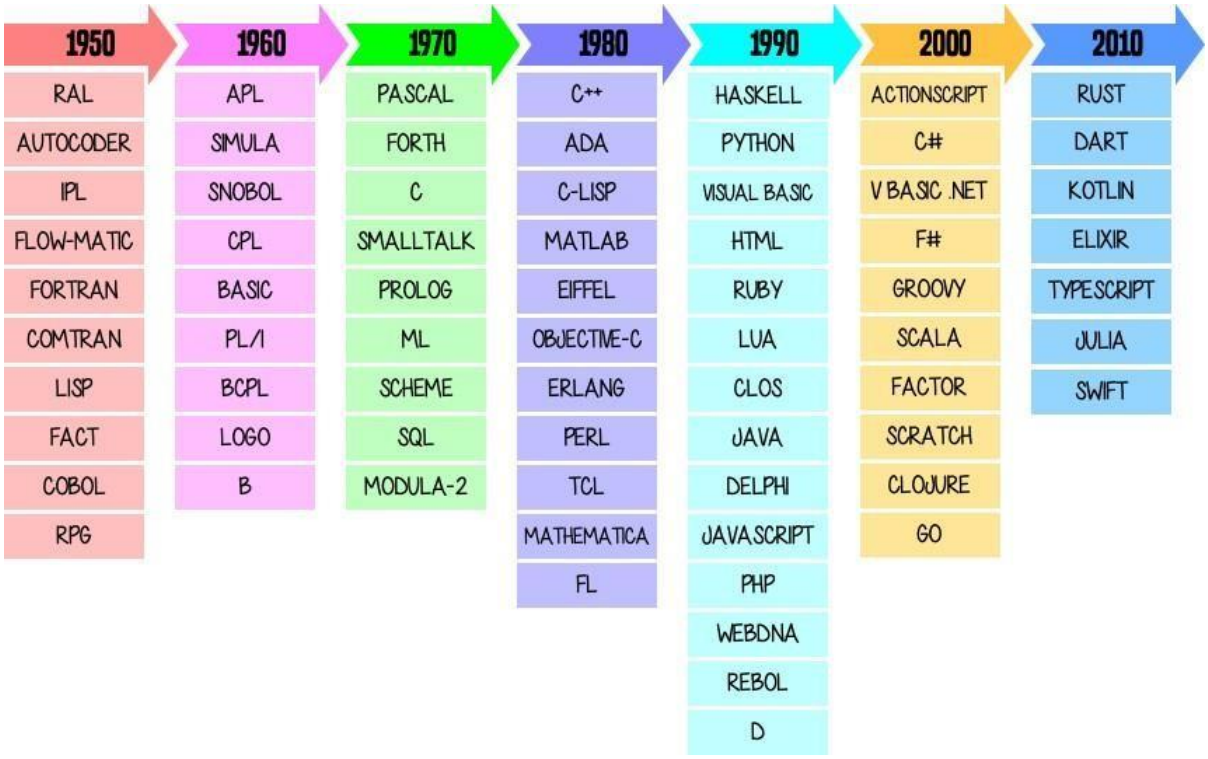


Figura 3: Cronología de los lenguajes de programación más destacados

1.3 Python



Python es un lenguaje de programación de *alto nivel* creado a finales de los 80/principios de los 90 por Guido van Rossum, holandés que trabajaba por aquella época en el *Centro para las Matemáticas y la Informática* de los Países Bajos. Sus instrucciones están muy cercanas al **lenguaje natural** en inglés y se hace hincapié en la **legibilidad** del código. Toma sunombre de los Monty Python, grupo humorista de los 60 que gustaban mucho a Guido. Python fue creado como sucesor del lenguaje ABC.[1]

1.3.1 Características del lenguaje

A partir de su definición de la Wikipedia:

- Python es un lenguaje de programación **interpretado** y **multiplataforma** cuya filosofía hace hincapié en una sintaxis que favorezca un **código legible**.
- Se trata de un lenguaje de programación **multiparadigma**, ya que soporta **orientación a objetos, programación imperativa** y, en menor medida, programación funcional.
- Añadiría, como característica destacada, que se trata de un lenguaje de **propósito general**.

[1] Foto original por Markéta Marcellová en Unsplash.

Ventajas

- Libre y gratuito (OpenSource).
- Fácil de leer, parecido a pseudocódigo.
- Aprendizaje relativamente fácil y rápido: claro, intuitivo….
- Alto nivel.
- Alta Productividad: simple y rápido.
- Tiende a producir un buen código: orden, limpieza, elegancia, flexibilidad, …
- Multiplataforma. Portable.
- Multiparadigma: programación imperativa, orientada a objetos, funcional, …
- Interactivo, modular, dinámico.
- Librerías extensivas («pilas incluídas»).
- Gran cantidad de librerías de terceros.
- Extensible (C++, C, …) y «embebible».
- Gran comunidad, amplio soporte.
- Interpretado.
- Tipado dinámico[5].
- Fuertemente tipado[6].
- Hay diferentes implementaciones: CPython, PyPy, Jython, IronPython, MicroPython, …

Desventajas

- Interpretado (velocidad de ejecución, multithread vs GIL, etc.).
- Consumo de memoria.
- Errores durante la ejecución.
- Dos versiones mayores no del todo compatibles (v2 vs v3).
- Desarrollo móvil.
- Documentación a veces dispersa e incompleta.

[5] Tipado dinámico significa que una variable puede cambiar de tipo durante el tiempo de vida de un programa. C es un lenguaje de tipado estático.

[6] Fuertemente tipado significa que, de manera nativa, no podemos operar con dos variables de tipos distintos, a menos que realice una conversión explícita. Javascript es un lenguaje débilmente tipado.

- Varios módulos para la misma funcionalidad.
- Librerías de terceros no siempre del todo maduras.

1.3.2 Uso de Python

Al ser un lenguaje de propósito general, podemos encontrar aplicaciones prácticamente en todos los campos científico-tecnológicos:

- Análisis de datos.
- Aplicaciones de escritorio.
- Bases de datos relacionales / NoSQL
- Buenas prácticas de programación / Patrones de diseño.
- Concurrencia.
- Criptomonedas / Blockchain.
- Desarrollo de aplicaciones multimedia.
- Desarrollo de juegos.
- Desarrollo en dispositivos embebidos.
- Desarrollo móvil.
- Desarrollo web.
- DevOps / Administración de sistemas / Scripts de automatización.
- Gráficos por ordenador.
- Inteligencia artificial.
- Internet de las cosas.
- Machine Learning.
- Programación de parsers / scrapers / crawlers.
- Programación de redes.
- Propósitos educativos.
- Prototipado de software.
- Seguridad.
- Tests automatizados.

De igual modo son muchas las empresas, instituciones y organismos que utilizan Python ensu día a día para mejorar sus sistemas de información. Veamos algunas de las más relevantes:



Figura 4: Grandes empresas y organismos que usan Python

Existen ránkings y estudios de mercado que sitúan a Python como uno de los lenguajes más *usados* y la vez, más *amados* dentro del mundo del desarrollo de software.

En el momento de la escritura de este documento, la última actualización del Índice TIOBE es de *febrero de 2022* en el que Python ocupaba el **primer lugar de los lenguajes de programación más usados**, por delante de *C* y *Java*.

Feb 2022	Feb 2021	Change	Programming Language	Ratings	Change
1	3	▲	Python	15.33%	+4.47%
2	1	▼	C	14.08%	-2.26%
3	2	▼	Java	12.13%	+0.84%

Figura 5: Índice TIOBE 2022

Igualmente en la encuesta a desarrolladores de Stack Overflow *hecha en 2021*, Python ocupaba el **tercer puesto de los lenguajes de programación más populares**, sólo por detrás de *Javascript* y *HTML/CSS*.



Figura 6: Encuesta Stack Overflow 2021

1.3.3 Python2 vs Python3

En el momento de la escritura de este material, se muestra a continuación la evolución delas versiones mayores de Python a lo largo de la historia:[3]

Versión	Fecha de lanzamiento
Python 1.0	Enero 1994
Python 1.5	Diciembre 1997
Python 1.6	Septiembre 2000
Python 2.0	Octubre 2000
Python 2.1	Abril 2001
Python 2.2	Diciembre 2001
Python 2.3	Julio 2003
Python 2.4	Noviembre 2004
Python 2.5	Septiembre 2006
Python 2.6	Octubre 2008
Python 2.7	Julio 2010
Python 3.0	Diciembre 2008
Python 3.1	Junio 2009
Python 3.2	Febrero 2011
Python 3.3	Septiembre 2012
Python 3.4	Marzo 2014
Python 3.5	Septiembre 2015
Python 3.6	Diciembre 2016
Python 3.7	Junio 2018
Python 3.8	Octubre 2019
Python 3.9	Octubre 2020
Python 3.10	Octubre 2021

El cambio de **Python 2** a **Python 3** fue bastante «traumático» ya que se **perdió la compatibilidad** en muchas de las estructuras del lenguaje. Los «*core-developers*»[4], con *Guido van Rossum* a la cabeza, vieron la necesidad de aplicar estas modificaciones en beneficio del rendimiento y expresividad del lenguaje de programación. Este cambio implicaba que el código escrito en Python 2 no funcionaría (de manera inmediata) en Python 3.

El pasado **1 de enero de 2020** finalizó oficialmente el **soporte a la versión 2.7** del lenguaje de programación Python. Es por ello que se recomienda lo siguiente:

Importante: Únete a **Python 3** y aprovecha todas sus ventajas.

1.3.4 Zen de Python

Existen una serie de *reglas* «filosóficas» que indican una manera de hacer y de pensar dentro del mundo **pitónico**[2] creadas por Tim Peters, llamadas el Zen de Python y que se pueden aplicar incluso más allá de la programación:

```
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly. Explicit is
better than implicit.Simple is better
than complex.
Complex is better than complicated.Flat is
better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.Although
practicality beats purity.
Errors should never pass silently.Unless
explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.Although that
way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.Namespaces
are one honking great idea -- let's do more of those!
```

En su traducción de la Wikipedia:

- Bello es mejor que feo.
- Explícito es mejor que implícito.
- Simple es mejor que complejo.
- Complejo es mejor que complicado.
- Plano es mejor que anidado.
- Espaciado es mejor que denso.

[2] Dícese de algo/alguien que sigue las convenciones de Python.

- La legibilidad es importante.
- Los casos especiales no son lo suficientemente especiales como para romper las reglas.
- Sin embargo la practicidad le gana a la pureza.
- Los errores nunca deberían pasar silenciosamente.
- A menos que se silencien explícitamente.
- Frente a la ambigüedad, evitar la tentación de adivinar.
- Debería haber una, y preferiblemente solo una, manera obvia de hacerlo.
- A pesar de que esa manera no sea obvia a menos que seas Holandés.
- Ahora es mejor que nunca.
- A pesar de que nunca es muchas veces mejor que *ahora* mismo.
- Si la implementación es difícil de explicar, es una mala idea.
- Si la implementación es fácil de explicar, puede que sea una buena idea.
- Los espacios de nombres son una gran idea, ¡tengamos más de esos!